BDF - Customized Product Documentation

BDF Racer2 V3 Product Documentation

1. System Overview for BDF Bakery Automation

The BDF Racer2 V3 system is a powerful AI Core service designed for enhanced line-scan acquisition, image stitching, live streaming, and optional AI-assisted inspection for your bakery automation processes. It ensures consistent product quality and efficient production monitoring.

1.1 Purpose

Racer2 V3 integrates with your production line to provide real-time visual data and analysis, supporting both hardware-backed and development workflows. It is designed to work with two Basler line cameras, capturing detailed views of your bakery products.

1.2 Core Capabilities for Production Monitoring

- **Dual-Camera Acquisition & Stitched Product View:** Captures images from two cameras and intelligently stitches them together to create a complete, unified view of each bakery item.
- **Browser-Friendly Monitoring:** Provides clear JPEG captures and live WebSocket streaming directly to your operator interface for immediate visual feedback.
- **Flexible Camera Access:** Supports direct (local), remote (via a separate service), or emulated (synthetic for testing) camera modes to fit various deployment needs.
- **Automated Quality Inspection:** Leverages optional RF-DETR or YOLO AI models for automated detection and tracking of bakery products, identifying deviations or quality concerns.
- **Tracked-Item Color Analysis:** Offers in-memory buffering and analysis of color data for tracked bakery items, aiding in consistency checks.
- **External System Integration:** Supports forwarding the latest product images to external inference services (`POST /analyze`) or running local AI inspections on pre-recorded images (`POST /inference`).

1.3 Operating Modes

The application can operate in three distinct camera modes to suit your BDF production environment:

- **`local`:** The main system directly controls the Basler cameras, ideal for setups where the processing unit is close to the hardware.
- **`remote`:** The main system fetches product images from a separate camera service over your network, useful for distributing workloads or when the camera host needs to remain dedicated to hardware access.

- **`emulated`:** The system generates synthetic images, perfect for testing the operator interface, API integrations, and AI workflows without requiring physical cameras.

AI processing offers flexibility:

- **Disabled:** For raw streaming and manual inspection only.
- **Enabled with `rfdetr`:** Utilizes RF-DETR for advanced detection and tracking.
- **Enabled with YOLO:** Provides YOLO detection combined with SORT tracking for comprehensive monitoring.

1.4 High-Level Production Flow

1. The main application (`main.py`) initiates the system and selects a camera runtime.

2. The camera runtime begins capturing, polling, or emulating JPEG images of your bakery products.

3. If AI inspection is active, model initialization proceeds in the background, reporting progress.

4. The most recent stitched product image is made available to:

- `GET /capture` (for current image retrieval)
- `POST /analyze` (for forwarding to external inference)
- `POST /inference` (for local file-based inspection)
- `WebSocket /ws` (for live streaming to the operator interface)

5. With color analysis active, samples from tracked bakery items are buffered and accessible via dedicated color endpoints, allowing for detailed consistency analysis.

1.5 System Components

- **`src/server`:** Manages application startup, API endpoints, camera mode selection, and AI status reporting.
- **`src/front`:** The browser-based operator interface and health dashboard.
- **`src/libs/racer2_camera`:** Handles camera acquisition, image stitching, resizing, and JPEG encoding for the live production workflow.
- **`src/libs/ai`:** Manages RF-DETR or YOLO inference, tracking of bakery items, annotation, and single-image inference capabilities.
- **`src/libs/color`:** Provides tools for color conversion, per-object analysis, and buffering of historical color samples for production insights.

2. System Architecture and Data Flow for BDF Production Monitoring

The BDF Racer2 V3 system is structured into layers to ensure robust and efficient operation, from camera acquisition to AI-driven quality control.

2.1 Runtime Layers

The system is logically divided into three layers:

1. **Entrypoints:** Initiates the main application (`main.py` on port `8000`) and the dedicated camera service (`camera_main.py` on port `8001`).

2. **Server Layer:** Exposes the main operator interface, live streaming, AI inspection, and color analysis endpoints (`src/server/app.py`). It also includes the host-side camera-only API (`src/server/camera_service.py`).

3. **Library Layer:** Contains reusable components for camera control (`src/libs/racer2_camera`), AI processing (`src/libs/ai`), and color analysis (`src/libs/color`).

2.2 Camera Pipeline for Quality Control

The live image acquisition and stitching process, critical for BDF's quality control, occurs as follows:

1. **Device Discovery:** Identifies Basler cameras using `pypylon`.

2. **Camera Requirements:** Ensures the configured number of cameras are available.

3. **Configuration:** Sets up trigger, pixel format, and optional white balance, unless `CAMERA_CONFIG_BY_PYLON=true` (indicating external camera configuration).

4. **Strip Acquisition:** Reads multiple vertical strips of pixels from each camera.

5. **Vertical Stacking:** Stacks these strips to form complete images from each camera.

6. **Alignment:** Applies horizontal and vertical offsets to align the images.

7. **Image Stitching:** Merges the output from multiple cameras into a single, comprehensive image of the product.

8. **Color Conversion:** Converts the stitched image to BGR format if required by the selected pixel format.

9. **Formatting:** Resizes and JPEG-encodes the final image for efficient consumption.

10. **Caching:** Stores the latest JPEG image for quick access by the operator interface and other systems.

Runtime adjustments to image stitching alignment can be performed via:

- `GET /camera/alignment`
- `POST /camera/alignment`

2.3 AI Pipeline for Automated Inspection

The AI orchestration is integral to BDF's automated inspection and quality assurance.

- **Model Selection:** The system uses either `rfdetr` for RF-DETR-backed detection and tracking, or YOLO detection with SORT tracking (for any other model name). This is controlled by `AI_MODEL_NAME`.
- **Asynchronous Initialization:** AI initialization runs in the background, preventing delays in server startup.

- **Status Monitoring:** `/status` provides updates on AI checkpoint progress, allowing operators to monitor readiness.
- **Live Stream:** The WebSocket stream sends raw product images until the AI is fully ready, then switches to AI-annotated frames.
- **Single-Image Inspection:** `run_single_image_inference()` supports `POST /inference` for analyzing individual product images from local storage.

Color Analysis for Product Consistency

The AI pipeline also manages a `ColorBuffer` that stores the history of tracked bakery items. This enables:

- **Per-Item Sample State:** Captures the latest color characteristics for each tracked product.
- **Rolling History:** Maintains a history of color samples for trend analysis.
- **Data Export:** Supports CSV export of color data for further analysis.
- **Production-Shaped Output:** Generates color analysis output tailored for downstream production monitoring systems.

2.4 Data Consumers for Integrated Monitoring

The latest processed product image is available to various monitoring and control systems:

- `GET /capture`: For retrieving the most recent JPEG image.
- `POST /analyze`: For forwarding images to external inference services.
- `POST /inference`: For local AI inference on specified image files.
- `WebSocket /ws`: For live streaming to the browser-based operator interface.
- The browser UI (`src/front/index.html`): Displays live and historical product images.

3. Configuration for BDF's Production Environment

The BDF Racer2 V3 system uses a clear and layered configuration approach to adapt to your specific production needs.

3.1 Configuration Sources

Configuration is managed through two types of files:

- **Non-Secret Runtime Settings:** Stored in [`settings.json`](../settings.json), these define operational parameters like camera modes, AI settings, and timing.
- **Secrets and Environment Selection:** Located in [`.env`](../.env) and optionally [`.env.debug`](../.env.debug), these files handle sensitive information like API keys and control the application's environment.

3.2 Loading Order (Prioritization)

The system loads configurations in a specific order, with later sources overriding earlier ones:

1. `.env` (base environment settings)

2. `APP_ENV_FILE` (if set, overrides `.env`) or `.env.debug` (if `APP_ENV=debug`)

3. `settings.json` (runtime settings, applied after environment files)

In practice for BDF:

- **Secrets:** Typically reside in `.env` for security.
- **Production-Specific Settings:** Non-secret parameters for different profiles (e.g., `production`, `debug`, `docker`) are best placed in `settings.json`.
- **Environment Variables:** Can override any setting, offering maximum flexibility when needed.

3.3 Key `settings.json` Parameters for BDF

Here are critical settings in `settings.json` that BDF operators and engineers might adjust:

Camera Runtime Configuration

- **`CAMERA_MODE`**: Defines how the system interacts with cameras:
- `local`: The main system initializes Basler cameras directly.
- `remote`: The main system polls a separate camera service over HTTP.
- `emulated`: Generates synthetic JPEG images for UI and API testing without physical cameras.
- **`CAMERA_WHITE_BALANCE`**: Enables one-shot white balance after image acquisition begins for consistent product coloring.
- **`CAMERA_TRIGGER_MODE`**: A boolean setting (`true` or `false`) to enable or disable camera triggering.
- **`CAMERA_PIXEL_FORMAT`**: Specifies how camera buffers are interpreted after stitching. Crucial for correct color representation of your bakery items:
- `BGR8`, `BGR8Packed`: These formats are treated as OpenCV-ready BGR images, skipping further color conversion. Use when the camera is already configured for BGR output.
- `YCbCr422_8`, `YUV422Packed`, `YUV422_YUYV_Packed`: These YUV422 formats are automatically converted to BGR for accurate display, JPEG encoding, streaming, and AI processing.
- **`CAMERA_CONFIG_BY_PYLON`**: When `true`, the system skips its own camera feature writes (pixel format, exposure, trigger, etc.) and uses the camera state already configured outside the application (e.g., via Pylon Viewer). Set to `false` if the application should control these settings.
- **`REMOTE_CAMERA_BASE_URL`**: Required when `CAMERA_MODE=remote`; specifies the address of the separate camera service.
- **`EMULATED_CAMERA_IMAGE_PATH`**: An optional image file used by the `emulated` runtime instead of generated frames for consistent test scenarios.

AI Runtime Configuration

- **`DETECTION_AND_TRACK`**: Enables or disables the AI detection and tracking functionality.
- **`AI_MODEL_NAME`**: Selects the AI model: `rfdetr` for RF-DETR, or another value for YOLO plus SORT tracking.
- **`YOLO_MODEL_PATH`**: Path to the YOLO model file if YOLO is selected.
- **`RFDETR_CONF_THRESH`**: Confidence threshold for RF-DETR detections.

3.4 `.env` File Parameters

These files hold sensitive API keys and environment selectors:

- `INFERENCE_API_HEADER` and `INFERENCE_API_KEY`: For authenticating with external inference services.
- `RFDETR_MODEL_ID` and `ROBOFLOW_API_KEY`: For configuring the RF-DETR model and Roboflow access.
- `APP_ENV`: Set to `debug` for enabling debug-specific settings.
- `APP_ENV_FILE`: Specifies an additional environment file to load, useful for different deployment profiles.

3.5 Recommended Practice for BDF

- **Security:** Keep all API keys and sensitive data exclusively in `.env` files.
- **Clarity:** Use `settings.json` for non-secret operational parameters that vary between production, debug, or Docker environments.
- **Debugging:** Use `APP_ENV=debug` to activate debug settings instead of modifying production values directly.
- **Docker Integration:** Reserve the `docker` profile in `settings.json` for container-specific settings that differ from your local or production host operations.

4. Deployment & Troubleshooting for BDF Operators

This section provides guidance for deploying and maintaining the BDF Racer2 V3 system, along with troubleshooting common operational issues.

4.1 Baseline Setup

Before any deployment, ensure these steps are completed:

- **Virtual Environment:** Create and activate a Python virtual environment.
- **Dependencies:** Install all required libraries using `pip install -r requirements.txt`.
- **Environment File:** Create your `.env` file from `.env.example` and populate it with relevant secrets and settings.
- **Review Settings:** Check `settings.json` for your desired operational parameters.
- **Profile Selection:** Confirm the intended application profile: `production`, `debug`, or `docker`.

4.2 Deployment Options for BDF Production

BDF can deploy the system in several ways, depending on hardware proximity and workload distribution.

Option 1: Single-Process Local Deployment

Use case: When the main processing machine has direct access to the Basler cameras and handles both image capture and AI inspection.

```powershell
python .\main.py
```

```

**Recommended settings:**

- `CAMERA_MODE=local`
- `DETECTION_AND_TRACK=true` or `false`, based on whether automated AI inspection is required.

#### Option 2: Debug or Hardware-Free Deployment

**Use case:** For testing the operator interface, API integrations, or AI workflows without requiring physical cameras.

```powershell

$env:APP_ENV="debug"

python .\main.py

```

Recommended settings:

- `CAMERA_MODE=emulated`
- Optionally set `EMULATED_CAMERA_IMAGE_PATH` to use a specific image file for consistent testing.

Option 3: Split Camera and Application Deployment

Use case: When the camera host needs to remain close to the Basler hardware, while the web interface and AI stack run on a separate machine.

Camera Host (e.g., on `http://192.168.1.100:8001`):

```powershell

python .\camera_main.py

```

This starts the camera-only service on port `8001`.

Main App Host (e.g., on `http://localhost:8000`):

Run the main application with:

- `CAMERA_MODE=remote`
- `REMOTE_CAMERA_BASE_URL=http://<camera-host-IP-or-hostname>:8001`

Then start:
```

```powershell

python .\main.py

```

Option 4: Docker for the Main App

Use case: To containerize the main application for consistent deployment, typically consuming images from a Windows host running the camera service.

Recommended pattern:

- A Windows host runs `camera_main.py` (Option 3: Camera Host).
- The Docker container runs the main application.
- The main app uses `CAMERA_MODE=remote`.
- Set `REMOTE_CAMERA_BASE_URL=http://host.docker.internal:8001` to allow the Docker container to reach the host machine's camera service.

Start with:

```powershell

docker compose up --build

```

4.3 Verification Checklist After Deployment

After deploying, perform these checks to ensure the system is operating correctly:

- Call `GET /status` (e.g., `http://localhost:8000/status`) and confirm it returns `200 OK`.
- Verify that `camera.ready` is `true` in the `status` response for your chosen camera mode.
- Check `GET /capture` (e.g., `http://localhost:8000/capture`) to confirm it returns a JPEG image.
- Open the browser interface on `/` (e.g., `http://localhost:8000/`) and confirm it loads and refreshes with product images.
- If AI is enabled, wait for `ai.phase` to become `ready` in the `status` response.
- If using `remote` mode, confirm the separate camera service on port `8001` also returns a healthy `/status`.

4.4 Common Troubleshooting Scenarios for BDF Operators

Server Starts But No Product Images Arrive

- **Check `GET /status`:** See if `camera.ready` is `false` or if the WebSocket is receiving `warming_up` messages.
- **Local Mode:**
- Verify pylon software is installed.
- Confirm both Basler cameras are connected and powered.
- Ensure the system has permissions to access the cameras.

- **Remote Mode:**
- Check `REMOTE_CAMERA_BASE_URL` for typos.
- Verify the host-side camera service (port `8001`) is running and accessible.
- Confirm `GET <REMOTE_CAMERA_BASE_URL>/capture` returns a JPEG image.
- **Emulated Mode:**
- Verify `CAMERA_MODE=emulated`.
- If `EMULATED_CAMERA_IMAGE_PATH` is set, ensure the file is readable.

Camera Busy Or Unavailable

- **Symptoms:** `/status` reports camera not ready, or logs mention camera initialization failure.
- **What to check:**
- Close Pylon Viewer or any other application that might be using the cameras.
- Confirm no other instance of the Racer2 V3 service is running.
- If issues persist, restart the camera host to reset the driver state.

Remote Camera Mode Does Not Refresh

- **What to check:**
- Confirm `REMOTE_CAMERA_BASE_URL` is correct.
- Ensure the camera host's port `8001` is not blocked by a firewall.
- Verify `GET <REMOTE_CAMERA_BASE_URL>/status` returns `200 OK`.
- Confirm the main app host can communicate with the camera host over the network (e.g., if Docker is used, `host.docker.internal` is resolving correctly).

WebSocket Stream Shows Text Instead Of Images

- This is expected when the camera is warming up, camera initialization failed, or `remote` mode has not yet received a valid product image. The text usually indicates camera status messages or `warming_up`.

AI Never Becomes Ready

- **Possible causes:** Incorrect Roboflow credentials, large model initialization time, or an unsupported local environment for AI.
- **What to verify:**
- Check `GET /status` for AI `phase` and `message`.
- Confirm `.env` contains valid AI credentials.
- Ensure the selected model in `AI_MODEL_NAME` is the intended one.

`POST /inference` Returns 404 (File Not Found)

- **What to check:**
- Verify the `SrcDir` path points to an existing directory on the machine running the main app.
- Confirm the `ImageName` is correct.
- Ensure the combined path `SrcDir/ImageName` points to an actual image file.

- **Remember:** This endpoint reads files from the local disk of the main app, not from remote URLs.

Color Endpoints Return Empty Data

- **Possible reasons:**
- `DETECTION_AND_TRACK=false` (color analysis depends on tracking).
- The AI model is not ready yet.
- No tracked items have been processed since the last buffer clear.
- A previous `GET /color-data` or `POST /color-analysis` call already cleared the buffer.

Pixel Format Problems (Wrong Colors, Conversion Errors)

- **Symptoms:** Product images show incorrect colors, or errors related to YUV conversion occur.
- **What to verify:**
- `CAMERA_PIXEL_FORMAT` in `settings.json` matches the actual output format of your cameras.
- `CAMERA_CONFIG_BY_PYLON=true` if external Pylon configuration manages the pixel format.
- Use `BGR8` if the camera is already configured to output OpenCV-ready BGR images.

General Diagnostic Sequence for Operators

1. **Check Status:** Call `GET /status` to get an overview of system health.
2. **Camera Readiness:** Confirm `camera.ready` is `true`.
3. **Image Capture:** Confirm `GET /capture` successfully returns a JPEG image.
4. **Browser UI:** Verify the browser interface updates with live product images.
5. **AI Readiness (if enabled):** Wait for `ai.phase=ready` in the `status` response.
6. **Functionality Tests:** Then test `/analyze`, `/inference`, or WebSocket streaming.